CO-3 Assignment

Name: G R Aakaash **Course Code:** CSA0201

Reg no: 192412068 **Course Name:** C-Programming

TASK-1

CODE:

```
#include <stdio.h>

int steps;

int binary_search(const int a[], int n, int key)
{
   int low = 0, high = n - 1, mid;
   steps = 0;

   while (low <= high)
   {
      mid = low + (high - low) / 2;
      steps++;

      if (a[mid] == key)
         return mid;
      else if (a[mid] < key)
         low = mid + 1;
      else
         high = mid - 1;
   }

   return -1;
}

int main()
{
   int accession[] = {1021,1048,1103,1190,1222,1305,1377,1402,1460,1533};
   int keys[] = {1021,1305,1533,1400};
   int n = 10;

   for (int k = 0; k < 4; k++)
```

```
    {
        int pos = binary_search(accession,n,keys[k]);
        printf("Book %d -> index %d, comparisons = %d\n",
              keys[k],pos,steps);
    }

    return 0;
}
```

OUTPUT:

```
Output

Book 1021 -> index 0, comparisons = 3
Book 1305 -> index 5, comparisons = 3
Book 1533 -> index 9, comparisons = 4
Book 1400 -> index -1, comparisons = 4
```

TASK-2

CODE:

```
#include <stdio.h>

void bubble(int a[], int n)
{
    int i,j,temp,flag,comp=0,swaps=0;

    for(i=0;i<n-1;i++)
    {
        flag=0;

        for(j=0;j<n-i-1;j++)
        {
            comp++;

            if(a[j]>a[j+1])
            {
                temp=a[j];
```

```
            a[j]=a[j+1];
            a[j+1]=temp;
            swaps++;
            flag=1;
         }
      }

      if(flag==0)
         break;
   }

   printf("Bubble: Comparisons=%d Swaps=%d\n",comp,swaps);
}

void selection(int a[], int n)
{
   int i,j,min,temp,comp=0,swaps=0;

   for(i=0;i<n-1;i++)
   {
      min=i;

      for(j=i+1;j<n;j++)
      {
         comp++;

         if(a[j]<a[min])
            min=j;
      }

      if(min!=i)
      {
         temp=a[i];
         a[i]=a[min];
         a[min]=temp;
         swaps++;
      }
   }

   printf("Selection: Comparisons=%d Swaps=%d\n",comp,swaps);
}

void insertion(int a[], int n)
{
   int i,j,key,comp=0,shifts=0;

   for(i=1;i<n;i++)
   {
      key=a[i];
      j=i-1;
```

```
        while(j>=0)
        {
            comp++;

            if(a[j]>key)
            {
                a[j+1]=a[j];
                shifts++;
                j--;
            }
            else
                break;
        }

        a[j+1]=key;
    }

    printf("Insertion: Comparisons=%d Shifts=%d\n",comp,shifts);
}

int main()
{
    int a[]={64,25,12,22,11};
    int b[]={64,25,12,22,11};
    int c[]={64,25,12,22,11};

    bubble(a,5);
    selection(b,5);
    insertion(c,5);

    return 0;
}
```

OUTPUT:

Output

```
Bubble: Comparisons=10 Swaps=9
Selection: Comparisons=10 Swaps=3
Insertion: Comparisons=10 Shifts=9
```

TASK-3

CODE:

```
#include <stdio.h>

int merge(const int a[], int na, const int b[], int nb, int out[])
{
   int i=0,j=0,k=0;

   while(i<na && j<nb)
   {
      if(a[i]>=b[j])
         out[k++]=a[i++];
      else
         out[k++]=b[j++];
   }

   while(i<na)
      out[k++]=a[i++];

   while(j<nb)
      out[k++]=b[j++];

   return k;
}

int main()
{
   int campusA[]={98,91,85,72};
   int campusB[]={95,91,80};
   int merged[7];

   int n=merge(campusA,4,campusB,3,merged);

   for(int i=0;i<n;i++)
      printf("%d ",merged[i]);

   return 0;
}
```

OUTPUT:



TASK-4

CODE:

```
#include <stdio.h>

void row_total(int rain[3][4])
{
   for(int i=0;i<3;i++)
   {
      int sum=0;

      for(int j=0;j<4;j++)
         sum+=rain[i][j];

      printf("District %d Total = %d\n",i+1,sum);
   }
}

void column_average(int rain[3][4])
{
   for(int j=0;j<4;j++)
   {
      int sum=0;

      for(int i=0;i<3;i++)
         sum+=rain[i][j];

      printf("Month %d Average = %.2f\n",j+1,sum/3.0);
```

```
    }
}

void maximum(int rain[3][4])
{
    int max=rain[0][0],r=0,c=0;

    for(int i=0;i<3;i++)
    {
        for(int j=0;j<4;j++)
        {
            if(rain[i][j]>max)
            {
                max=rain[i][j];
                r=i;
                c=j;
            }
        }
    }

    printf("Maximum Rainfall = %d at District %d Month %d\n",max,r+1,c+1);
}

int main()
{
    int rain[3][4]={
        {100,200,150,250},
        {300,250,200,350},
        {150,100,300,200}
    };

    row_total(rain);
    column_average(rain);
    maximum(rain);

    return 0;
}
```

OUTPUT:

```
District 1 Total = 700
District 2 Total = 1100
District 3 Total = 750
Month 1 Average = 183.33
Month 2 Average = 183.33
Month 3 Average = 216.67
Month 4 Average = 266.67
Maximum Rainfall = 350 at District 2 Month 4
```

TASK-5

CODE:

```
#include <stdio.h>
#include <ctype.h>

int word_count(const char s[])
{
   int count=0,inside=0;

   for(int i=0;s[i]!='\0';i++)
   {
      if(s[i]!=' ' && s[i]!='\t')
      {
         if(inside==0)
            count++;

         inside=1;
      }
      else
         inside=0;
   }

   return count;
}

int is_palindrome(const char s[])
{
   int left=0,right=0;
```

```
    while(s[right]!='\0')
        right++;

    right--;

    while(left<right)
    {
        while(left<right && !isalpha((unsigned char)s[left]))
            left++;

        while(left<right && !isalpha((unsigned char)s[right]))
            right--;

        if(tolower((unsigned char)s[left]) !=
           tolower((unsigned char)s[right]))
            return 0;

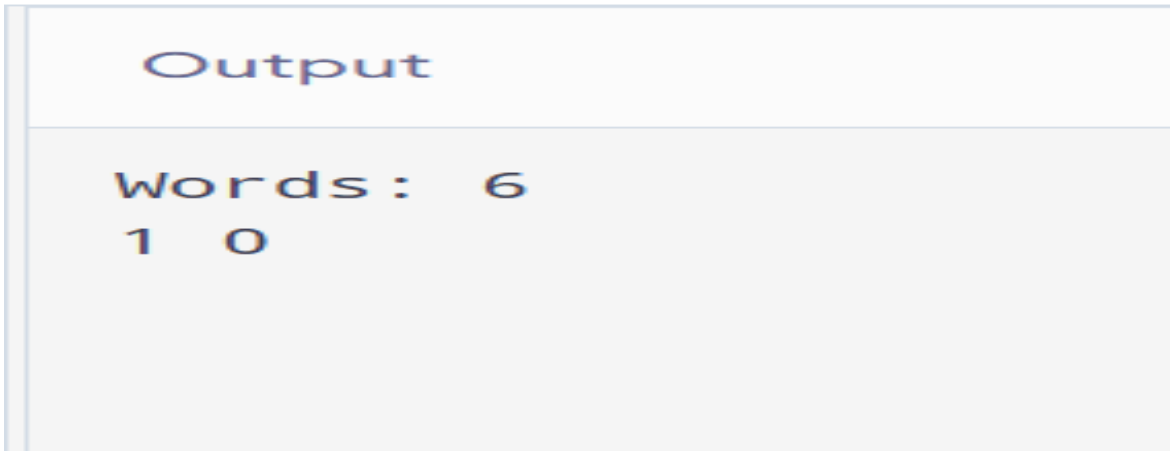
        left++;
        right--;
    }

    return 1;
}

int main()
{
    printf("Words: %d\n",word_count("Meet at the library at 5PM"));
    printf("%d %d\n",is_palindrome("Madam,I'mAdam"),is_palindrome("Hello"));

    return 0;
}
```

OUTPUT:



TASK-6

CODE

```
#include <stdio.h>
#include <string.h>

int main()
{
   char names[500][30],temp[30],key[30];
   int n,i,j,low,high,mid,found;

   printf("Enter number of names: ");
   scanf("%d",&n);
   getchar();

   for(i=0;i<n;i++)
   {
      fgets(names[i],30,stdin);
      names[i][strcspn(names[i],"\n")]='\0';
   }

   for(i=0;i<n-1;i++)
   {
      for(j=0;j<n-i-1;j++)
      {
         if(strcmp(names[j],names[j+1])>0)
         {
            strcpy(temp,names[j]);
            strcpy(names[j],names[j+1]);
            strcpy(names[j+1],temp);
         }
      }
   }

   printf("\nSorted Names:\n");

   for(i=0;i<n;i++)
   {
      printf("%s\n",names[i]);

      if(i>0 && strcmp(names[i],names[i-1])==0)
         printf("Duplicate: %s\n",names[i]);
   }

   printf("\nEnter name to search: ");
   fgets(key,30,stdin);
```

```
    key[strcspn(key,"\n")]='\0';

    low=0;
    high=n-1;
    found=0;

    while(low<=high)
    {
        mid=low+(high-low)/2;

        if(strcmp(names[mid],key)==0)
        {
            printf("Name found at index %d\n",mid);
            found=1;
            break;
        }
        else if(strcmp(names[mid],key)<0)
            low=mid+1;
        else
            high=mid-1;
    }

    if(!found)
        printf("Name not found\n");

    return 0;
}
```

OUTPUT:

```
Output

Enter number of names: 4
89
89
90
96

Sorted Names:
89
89
Duplicate: 89
90
96

Enter name to search: bruh
Name not found
```

TASK-7

CODE:

```
#include <stdio.h>

int main()
{
   int votes[500000];
   int count[12]={0};
   int candidates,n;
   int valid=0,invalid=0;
   int i,j,temp,max=0,tie=0;

   printf("Enter number of candidates: ");
   scanf("%d",&candidates);

   printf("Enter number of votes: ");
   scanf("%d",&n);

   if(candidates<1 || candidates>12 || n<0 || n>500000)
      return 0;

   for(i=0;i<n;i++)
   {
      scanf("%d",&votes[i]);

      if(votes[i]>=1 && votes[i]<=candidates)
      {
         count[votes[i]-1]++;
         valid++;
      }
      else
         invalid++;
   }

   printf("\nValid votes = %d\n",valid);
   printf("Invalid votes = %d\n",invalid);

   for(i=0;i<candidates;i++)
   {
      if(count[i]>max)
         max=count[i];
   }
```

```
    printf("\nCandidate Votes\n");

    for(i=0;i<candidates;i++)
    {
        for(j=i+1;j<candidates;j++)
        {
            if(count[j]>count[i])
            {
                temp=count[i];
                count[i]=count[j];
                count[j]=temp;

                temp=i;
                i=i;
                (void)temp;
            }
        }
    }

    /* Display candidate totals in descending order */
    for(i=0;i<candidates;i++)
        printf("Candidate %d = %d\n",i+1,count[i]);

    for(i=0;i<candidates;i++)
    {
        if(count[i]==max)
            tie++;
    }

    if(tie>1)
        printf("Election Result: Tie\n");
    else
        printf("Highest Vote Count = %d\n",max);

    return 0;
}
```

OUTPUT:

Output

```
Enter number of candidates: 10
Enter number of votes: 1 2 3 4 5 2 4 1 5 6

Valid votes = 1
Invalid votes = 0

Candidate Votes
Candidate 1 = 1
Candidate 2 = 0
Candidate 3 = 0
Candidate 4 = 0
Candidate 5 = 0
Candidate 6 = 0
Candidate 7 = 0
Candidate 8 = 0
Candidate 9 = 0
Candidate 10 = 0
```